
Agentes Pilotos para ANTIGRAVITY: Neuroevolución Determinista de Políticas Robustas en un Entorno Multi-Agente con Caos No Estacionario

Juan Manuel Prieto Corredor
Universidad de La Sabana
juanprico@unisabana.edu.co

Slendy M. Grisales Rueda
Universidad de La Sabana
slendygrru@unisabana.edu.co

Manuel Alejandro Niño Leon
Universidad de La Sabana
manuelnile@unisabana.edu.co

Abstract

Diseñamos AG-RACE, un entorno determinista (server-authoritative) tipo Gymnasium para carreras multi-agente en pistas procedurales con eventos caóticos (viento, fricción variable, hazards y debuffs como control invertido, stun o ruido sensorial). Entrenamos políticas neuronales compactas mediante neuroevolución (ES/ARS, CMA-ES) y coevolución/self-play con pools de oponentes, buscando generalización por semillas y robustez bajo cambios de distribución (intensidad de caos). Proponemos una representación estructurada (sensores de pista + rivales + eventos + parámetros del mundo), una arquitectura multi-habilidad con mezcla por compuerta (driving vs overtaking), y una función de fitness multi-objetivo con regularización por varianza inter-seed. Evaluamos en 100 semillas no vistas, reportando desempeño competitivo, seguridad (colisiones/km y off-track), robustez a debuffs y latencia de inferencia (TorchScript/ONNX) con objetivo < 10 ms p95.

1 Introducción

La conducción autónoma en simulación es un banco de pruebas natural para estudiar control continuo, toma de decisiones bajo incertidumbre y aprendizaje competitivo. Sin embargo, dos aspectos elevan significativamente la dificultad: (i) **generación procedural** de pistas, donde el agente debe generalizar a geometrías no vistas, y (ii) **no estacionariedad multi-agente**, donde oponentes adaptativos y eventos adversos cambian la distribución efectiva de estados en tiempo de ejecución.

En este proyecto investigamos si la **neuroevolución**—optimización de parámetros como caja negra— puede inducir políticas robustas en un entorno determinista condicionado por semilla. A diferencia de enfoques puramente por gradiente, ES/ARS y CMA-ES permiten optimizar directamente comportamientos en presencia de discontinuidades (p. ej., colisiones) y lógica de juego (trampas/debuffs) sin depender de estimadores de gradiente potencialmente inestables.

Objetivos. Siguiendo el documento base, establecemos cuatro objetivos: **O1 (Entorno)**: implementar un env Gymnasium multi-agente, server-authoritative, con step fijo (60Hz) e inferencia configurable (10–30Hz), determinista por seed. **O2 (Política)**: diseñar una red ligera con observaciones estructuradas y acciones continuas con capa de seguridad. **O3 (Entrenamiento)**: entrenar con neuroevolución (ES/ARS, CMA-ES) y/o coevolución. **O4 (Evaluación)**: comparar contra un baseline heurístico en 100 seeds y reportar desempeño, robustez y latencia.

Contribución central. Estudiar si una política compacta con representación estructurada puede aprender principios transferibles de conducción en un sistema dinámico no estacionario, manteniendo seguridad, estabilidad y competitividad.

2 Trabajo relacionado

ES/ARS y neuroevolución. ES como alternativa escalable al RL por gradiente ha sido ampliamente discutida por Salimans et al. [6]. CMA-ES es un estándar práctico para optimización continua no convexa y suele destacar cuando el número de parámetros es moderado [3].

Control competitivo en carreras. En entornos tipo TORCS, se observó que la neuroevolución puede producir controladores competitivos separando habilidades (conducción vs adelantamiento) y combinándolas luego en un controlador final [2]. Este patrón inspira explícitamente nuestra arquitectura multi-habilidad.

Multi-agente y coevolución. La coevolución y el self-play generan currículos emergentes pero introducen riesgo de inestabilidad y ciclos. Klijn y Eiben proponen enfoques coevolutivos profundos en setting multi-agente [4]. Adicionalmente, la literatura sobre autocurrículos multi-agente muestra cómo la competencia puede inducir progresión de habilidades (aunque en un dominio distinto) [1].

Modelos generativos de comportamiento. Aunque nuestro trabajo no es generativo, STEVE-1 muestra una línea emergente: modelos de comportamiento (policy/behavior modeling) que se benefician de preentrenamiento y adaptación [5]. Esto es relevante como dirección futura (p. ej., distillation o pretraining de conductas) para mejorar generalización.

3 Formulación del problema

Modelamos AG-RACE como un juego estocástico parcialmente observable con **dinámica determinista condicionada por semilla** ω .

Definición. Sea N el número de agentes. En el tiempo t , el entorno tiene estado global $s_t \in \mathcal{S}$ y cada agente i observa $o_t^i \in \mathcal{O}$ mediante una función de observación $o_t^i = \Omega(s_t, i)$. Cada agente ejecuta acción continua $a_t^i \in \mathcal{A} \subset \mathbb{R}^5$.

La transición es determinista dado ω :

$$s_{t+1} = f_\omega(s_t, a_t^1, \dots, a_t^N). \quad (1)$$

Las recompensas son igualmente deterministas condicionadas por ω :

$$r_t^i = R_\omega^i(s_t, a_t^1, \dots, a_t^N). \quad (2)$$

Objetivo de optimización. Buscamos parámetros $\theta \in \mathbb{R}^p$ que maximizan fitness esperado sobre semillas y oponentes:

$$\max_{\theta} J(\theta) = \mathbb{E}_{\omega \sim \mathcal{D}} \mathbb{E}_{\text{opp} \sim \mathcal{P}} [F(\tau_{\omega, \text{opp}}; \theta)], \quad (3)$$

donde $\mathcal{P}$ es una distribución sobre oponentes (pool histórico + baselines), y $\tau_{\omega, \text{opp}}$ es la trayectoria conjunta en el episodio.

4 Entorno AG-RACE: diseño, determinismo y caos

4.1 Diseño server-authoritative y reproducibilidad

El entorno es **autoritativo**: colisiones, ranking, efectos de hazards y debuffs se resuelven en el servidor (el env). Para reproducibilidad, registramos:

- semilla ω y estados del RNG;
- parámetros del mundo (fricción, gravedad, viento, visibilidad);

- especificación procedural de pista (hash/serialización);
- schedule de eventos (tipos, tiempos, severidad);
- versión del código/config (hash de configuración).

4.2 Frecuencias: física vs inferencia

Usamos step físico fijo a 60 Hz. La inferencia del controlador se ejecuta a $f_\pi \in [10, 30]$ Hz. Entre ticks de inferencia, mantenemos la acción (action hold) o interpolamos suavemente si se requiere estabilidad.

4.3 Generación procedural de pista

Representamos la pista por una línea central $\gamma : [0, L] \rightarrow \mathbb{R}^2$ (o $\mathbb{R}^3$ si hay elevación) y un perfil de ancho $w(s)$. El progreso escalar del agente se define proyectando la posición del vehículo sobre γ :

$$s_t = \arg \min_{s \in [0, L]} \|\mathbf{p}_t - \gamma(s)\|_2, \quad \Delta s_t = s_{t+1} - s_t. \quad (4)$$

Esto permite recompensar avance independientemente de geometría exacta.

4.4 Eventos caóticos y debuffs

Definimos el conjunto de eventos activos:

$$E_\omega(t) = \{e_t^{(1)}, \dots, e_t^{(k_t)}\}, \quad (5)$$

donde cada evento incluye tipo $c \in \{1, \dots, C\}$, severidad $\rho \in [0, 1]$, y duración restante $d \in \mathbb{N}$. Los debuffs se modelan como variables de estado con decaimiento determinista:

$$d_{t+1} = \max(0, d_t - 1). \quad (6)$$

Ejemplos: control invertido, stun breve, ruido sensorial, drift impairment.

4.5 API estilo Gymnasium

El entorno expone:

- `reset(seed, world_params) → obs_dict`
- `step(action_dict) → obs_dict, reward_dict, terminated, truncated, info`

`info` incluye señales para evaluación: ranking, flags de colisión/off-track/stuck, y telemetría para métricas.

5 Observaciones y acciones

5.1 Observación estructurada (MVP, 85 dims)

Cada agente recibe un vector estructurado (base del documento), con dimensión aproximada 85:

- **Sensores de pista (33)**: 31 raycasts normalizados $[0, 1] + \text{heading_error, lateral_error}$.
- **Estado propio (10)**: velocidad/aceleración normalizadas, $\sin / \cos(\text{yaw})$, yaw_rate , slip , progreso s , flags.
- **Rivales (18)**: $K = 3$ rivales $\times 6$ dims (posición/vel relativa + $\sin / \cos$ heading).
- **Eventos/Hazards (14)**: top- $M = 2$ con embedding de tipo (dim 4) + (dist, ETA, severidad).
- **Mundo+debuffs (10)**: 4 params del mundo + 3 debuffs (flag+tiempo).

Table 1: Especificación de observación (MVP, basada en el diseño del documento).

Bloque	Descripción	Dims
Track sensors	31 raycasts + 2 errores geométricos	33
Self-state	cinemática + flags	10
Rivales	$K = 3$ con 6 dims por rival	18
Eventos	$M = 2$, emb(4) + (dist, ETA, sev)	14
Mundo+debuffs	4 params + 3 debuffs(flag+time)	10
Total		85

5.2 Acciones continuas

Acciones (MVP):

$$a = [\text{throttle}, \text{brake}, \text{steer}, \text{drift}, \text{stabilize}] \in [-1, 1]^5. \quad (7)$$

Aplicamos **clipping** y una **capa de seguridad** antes de ejecutar.

6 Política neuronal compacta (arquitectura cerrada)

La compactación es clave para viabilizar CMA-ES y mantener exploración eficiente. Apuntamos a $\sim 12k$ parámetros.

6.1 Embeddings de eventos

Para tipo de evento $t \in \{1, \dots, 8\}$:

$$E_{\text{event}}(t) \in \mathbb{R}^4. \quad (8)$$

Esto permite generalizar por categorías de hazard con bajo costo paramétrico.

6.2 Encoder de rivales + pooling

Para cada rival i con vector $r_i \in \mathbb{R}^6$:

$$h_i = \tanh(W_2 \tanh(W_1 r_i + b_1) + b_2) \in \mathbb{R}^{16}. \quad (9)$$

Agregamos con max-pooling (invariante a permutaciones):

$$h_{\text{pool}} = \max_i h_i \in \mathbb{R}^{16}. \quad (10)$$

6.3 Tronco compartido + dos cabezas (multi-skill)

Concatenamos:

$$z_0 = [x_{\text{track}}, x_{\text{self}}, h_{\text{pool}}, x_{\text{events}}, x_{\text{world}}]. \quad (11)$$

Luego:

$$z_1 = \tanh(W_3 z_0 + b_3) \in \mathbb{R}^{64}, \quad z_2 = \tanh(W_4 z_1 + b_4) \in \mathbb{R}^{48}. \quad (12)$$

Dos cabezas:

$$u^{(d)} = W_6 \tanh(W_5 z_2 + b_5) + b_6 \in \mathbb{R}^3, \quad (13)$$

$$u^{(o)} = W_8 \tanh(W_7 z_2 + b_7) + b_8 \in \mathbb{R}^3. \quad (14)$$

6.4 Gate: mezcla suave de habilidades

Definimos:

$$\phi = [\text{min_front_gap}, \text{nearest_rival_dist}, \text{rel_speed}] \in \mathbb{R}^3, \quad g = \sigma(w_g^\top \phi + b_g). \quad (15)$$

Acción raw:

$$u = (1 - g)u^{(d)} + gu^{(o)}. \quad (16)$$

6.5 Rangos de salida

$$\text{steer} = \tanh(u_1) \in [-1, 1], \quad \text{throttle} = \sigma(u_2) \in [0, 1], \quad \text{brake} = \sigma(u_3) \in [0, 1]. \quad (17)$$

6.6 Capa de seguridad y fallback heurístico

Aplicamos reglas deterministas:

- Si $\text{min_front_gap} < 0.12$ y $\text{throttle} > 0.35$: forzar $\text{brake} \geq 0.6$ y $\text{throttle} \leq 0.1$.
- Si riesgo off-track alto: limitar $|\text{steer}| \leq 0.6$.
- Si stuck_timer alto: modo recuperación (throttle suave + steer hacia el centro).
- Si riesgo alto persiste: activar baseline por ventana corta (p.ej. 10 ticks).

7 Recompensa y función de fitness multi-objetivo

7.1 Recompensa por tick (shaping)

La recompensa por tick combina avance, competitividad y seguridad:

$$r_t = \alpha \Delta s_t + \beta \Delta \text{rank}_t + \kappa \mathbb{I}[\text{checkpoint}] + \mu \mathbb{I}[\text{lap_complete}] - \lambda_c \mathbb{I}[\text{collision}] - \lambda_o \mathbb{I}[\text{offtrack}] - \lambda_s \mathbb{I}[\text{stuck}] - \lambda_z Z_t + \eta H_t. \quad (18)$$

Z_t penaliza oscilación (zigzag) sin avance y H_t recompensa supervivencia/evitación de hazards (o recuperación rápida).

7.2 Retorno por episodio

$$R_\omega(\theta) = \sum_{t=1}^T r_t. \quad (19)$$

7.3 Fitness multi-seed + estabilidad (varianza inter-seed)

Promediamos sobre un conjunto de semillas y oponentes y penalizamos varianza inter-seed:

$$J(\theta) = \mathbb{E}_{\omega, \text{opp}} \left[\lambda_1 R_\omega(\theta) + \lambda_2 \text{Rank}_\omega(\theta) - \lambda_3 \text{Coll}_\omega(\theta) - \lambda_4 T_{\text{off}}^\omega \right] - \lambda_v \text{Var}_\omega(R_\omega(\theta)). \quad (20)$$

Justificación del término de varianza. Sin penalización de varianza, la búsqueda evolutiva puede converger a políticas con alto rendimiento medio pero frágiles a ciertos seeds (fallas catastróficas). Penalizar Var_ω fuerza soluciones más consistentes, alineadas con el objetivo de generalización.

Normalización y elección de pesos. Recomendamos normalizar cada término por su escala típica (p.ej., colisiones por km, off-track como fracción del episodio) y ajustar λ_i en validación fija para evitar que un solo término domine el fitness.

8 Entrenamiento por neuroevolución y coevolución

8.1 ES/ARS

Optimizamos θ con perturbaciones $\epsilon_k \sim \mathcal{N}(0, I)$:

$$\hat{g} = \frac{1}{K\sigma} \sum_{k=1}^K (F(\theta + \sigma\epsilon_k) - F(\theta - \sigma\epsilon_k)) \epsilon_k, \quad \theta \leftarrow \theta + \alpha \hat{g}. \quad (21)$$

ES es altamente paralelizable por rollouts [6].

8.2 CMA-ES

CMA-ES adapta la covarianza de exploración, especialmente útil para políticas compactas [3]. En práctica, consideramos variante diagonal si el costo de covarianza completa se vuelve prohibitivo.

8.3 Coevolución / self-play estabilizado

Mantenemos un **opponent pool** $\mathcal{P}$ con élites históricas y baselines. La evaluación de candidatos muestrea oponentes de $\mathcal{P}$ para reducir inestabilidad y evitar sobreajuste a un único adversario [4].

8.4 Currículo (recomendado)

Tres fases (basadas en el plan del documento):

1. Fase 1: conducción sin rivales.
2. Fase 2: adelantamiento con 1 rival scripted/pool fijo.
3. Fase 3: mezcla + coevolución (self-play con pool).

Esto sigue la intuición de separación de habilidades competitivas [2].

9 Baseline heurístico (referencia y fallback)

Implementamos un baseline determinista:

- seguimiento de línea ideal (pure pursuit) con lookahead fijo;
- control P/PID ligero sobre error lateral y de rumbo;
- modulación de velocidad por curvatura local y espacio libre (raycasts);
- evasión reactiva: frenado preventivo + sesgo de steer hacia el lado con mayor espacio;
- reglas deterministas de drift/estabilización por umbrales.

Se usa como referencia mínima y fallback de seguridad.

10 Protocolo experimental y métricas

10.1 Split de seeds y evaluación

Particionamos seeds en train/val/test:

- **Train:** seeds usados para optimización.
- **Val:** seeds fijos para selección de hiperparámetros y early stopping.
- **Test:** 100 seeds no vistas para reporte final.

10.2 Evaluación out-of-distribution (OOD)

En test ejecutamos un **shift** de distribución aumentando intensidad/frecuencia de caos (viento/fricción/debuffs) para medir robustez.

10.3 Métricas

Reportamos:

- **Rendimiento:** recompensa media, ranking medio, % victorias/top-3.
- **Seguridad:** colisiones por km, fracción de tiempo off-track, episodios terminados por stuck.
- **Robustez:** degradación relativa bajo OOD (caos aumentado), sensibilidad a debuffs.
- **Estabilidad:** varianza inter-seed de R_ω y (opcional) varianza inter-oponente.
- **Latencia:** p50/p95 de inferencia en TorchScript/ONNX, objetivo < 10 ms p95.
- **Determinismo:** consistencia por seed (bitwise o tolerancia numérica fija).

Table 2: Plantilla de resultados principales (completar con valores reales).

Método	Return $\uparrow$	Rank $\downarrow$	Coll/km $\downarrow$	Lat p95 (ms) $\downarrow$
Baseline heurístico	TBD	TBD	TBD	TBD
ES/ARS (ours)	TBD	TBD	TBD	TBD
CMA-ES (ours)	TBD	TBD	TBD	TBD
CMA-ES + coevolución	TBD	TBD	TBD	TBD

Table 3: Plantilla OOD: degradación bajo caos aumentado.

Método	Δ Return(%) $\uparrow$	Δ Coll/km(%) $\downarrow$	Δ Rank(%) $\downarrow$
ES/ARS	TBD	TBD	TBD
CMA-ES	TBD	TBD	TBD
CMA-ES + coevol.	TBD	TBD	TBD

10.4 Ablations

Aislamos contribuciones:

1. Sensores: raycasts vs grid polar.
2. Representación: sin embeddings de evento (one-hot).
3. Rivales: sin pooling (concatenación cruda).
4. Seguridad: sin capa de seguridad / sin fallback.
5. Entrenamiento: sin coevolución (oponentes fijos).

11 Reproducibilidad

Checklist propuesto:

- Dockerfile y versiones fijadas.
- Configs (YAML) versionadas: f_π , K , M , pesos λ .
- Seeds registradas: train/val/test.
- Hash de configuración por corrida.
- Logs: telemetría por tick (colisión/off-track/stuck) + latencia.
- Script único de evaluación para reproducir tablas.

12 Discusión

El diseño integra tres tensiones: (i) expresividad vs explorabilidad (política compacta para CMA-ES), (ii) competitividad vs seguridad (capa de seguridad + fallback), y (iii) rendimiento promedio vs robustez (penalización por varianza inter-seed). La coevolución aporta currículo emergente, pero introduce no estacionariedad; el opponent pool amortigua este efecto.

Como dirección futura, técnicas de modelado de comportamiento (p.ej., preentrenamiento o distillation) podrían mejorar generalización o reducir costo de rollouts [5]. Además, evidencia sobre autocurrículos en multi-agente sugiere que objetivos simples pueden inducir progresión de habilidades, lo cual podría inspirar currícula más sistemáticos [1].

13 Limitaciones

- **Ineficiencia muestral:** neuroevolución requiere muchos episodios.
- **Escalabilidad:** CMA-ES completo escala mal con p ; puede requerir variantes diagonales.

- **Sensibilidad al shaping:** pesos λ pueden inducir comportamientos degenerados si se calibran mal.
- **No estacionariedad:** coevolución puede ciclar; el pool reduce pero no elimina el riesgo.
- **Sim-to-real:** robustez en simulación no garantiza transferencia a dinámicas reales.

14 Plan de trabajo (12 semanas)

- S1: Setup repo + Docker + API final + seeding/logging.
- S2: Pista procedural determinista + checkpoints + progreso s .
- S3: Física step fijo 60Hz + params (fricción/gravedad/viento).
- S4: Sensores + hazards/eventos deterministas + unit tests.
- S5: Multi-agente + colisiones autoritativas + ranking determinista.
- S6: Baseline + safety fallback + instrumentación de métricas.
- S7: Política MLP + normalización de obs + sanity checks.
- S8: Export TorchScript/ONNX + benchmark latencia p50/p95.
- S9: ES/ARS MVP + logging + superar baseline en 20 seeds.
- S10: CMA-ES (o diagonal) + comparación justa.
- S11: Robustez al caos + coevolución + evaluación OOD.
- S12: Evaluación final 100 seeds + ablations + reporte.

15 Conclusión

Presentamos un marco determinista y reproducible para estudiar neuroevolución en carreras multi-agente con caos no estacionario. La combinación de observación estructurada, política compacta multi-skill, capa de seguridad y fitness multi-seed busca maximizar generalización, robustez y viabilidad de despliegue (latencia). El protocolo experimental prioriza evaluación en 100 seeds no vistas y análisis OOD, permitiendo conclusiones defendibles sobre transferencia y estabilidad.

References

- [1] Bowen Baker, Ingmar Kanitscheider, Todor M. Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. Emergent tool use from multi-agent autocurricula. *arXiv preprint arXiv:1909.07528*, 2019.
- [2] Luigi Cardamone, Daniele Loiacono, and Pier Luca Lanzi. Evolving competitive car controllers for racing games with neuroevolution. In *Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation (GECCO)*, pages 1179–1186. ACM, 2009.
- [3] Nikolaus Hansen. The cma evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.
- [4] Daan Klijn and A. E. Eiben. A coevolutionary approach to deep multi-agent reinforcement learning. *arXiv preprint arXiv:2104.05610*, 2021.
- [5] Shalev Lifshitz, Keiran Paster, Harris Chan, Jimmy Ba, and Sheila McIlraith. Steve-1: A generative model for text-to-behavior in minecraft. *arXiv preprint arXiv:2306.00937*, 2023.
- [6] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.

A Pseudocódigo del entorno

B Detalles de latencia

Benchmark: medir p50/p95 en batch=1, incluyendo preprocesado, para TorchScript y ONNX.

Algorithm 1 AG-RACE: reset y step (resumen)

reset(ω , world_params):
Inicializar RNG con ω
Generar pista procedural, checkpoints, hazards y schedule de eventos
Inicializar agentes y telemetría
Retornar obs_dict

step($\{a^i\}_{i=1}^N$):
Aplicar capa server-authoritative: colisiones, hazards, debuffs
Avanzar física a 60Hz (posible action-hold si $f_\pi < 60$)
Computar progreso Δs , ranking, flags de seguridad
Construir obs_dict, reward_dict, info
Retornar transición
